

SafeCircle

Redesigned Reading Edition

A cleaned, structured version of the uploaded conversation document, reorganized for readability while preserving the original substance.

Contents

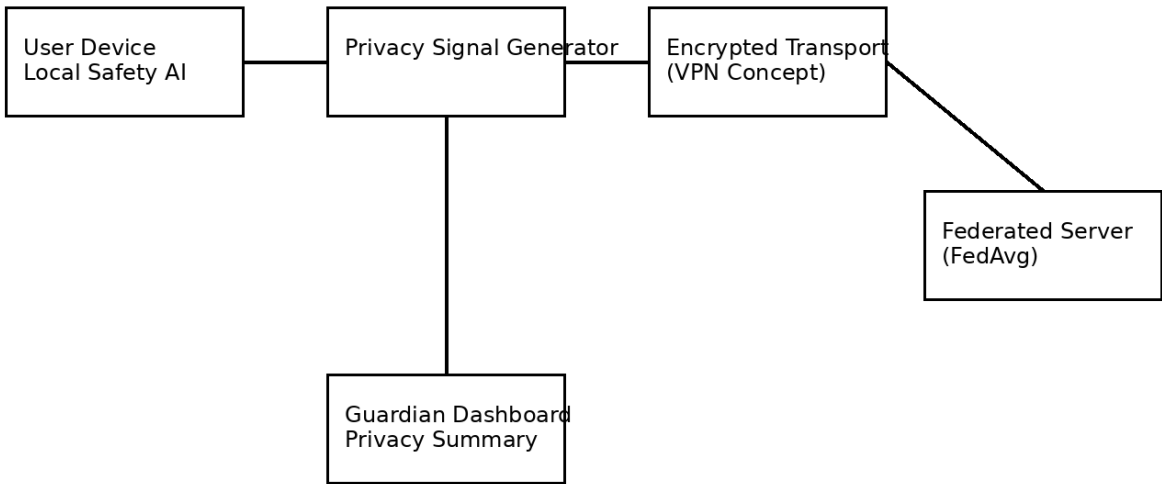
- 1. What Judges Want
- 2. Stronger Project Idea
- 3. ExpressVPN Positioning
- 4. Product Features and Reporting Flow
- 5. Frontend and Demo Story
- 6. Pitch and Winning Framing
- 7. SDK Blueprint
- 8. README and Submission Notes

Designed for reading, review, and hackathon preparation.

Architecture Overview

This cleaner diagram summarizes the core product story from the source document: on-device safety analysis, privacy-preserving signal sharing, encrypted transport, federated coordination, and privacy-aware guardian guidance.

SafeCircle Architecture



Demo Interface Snapshot

This visual mockup captures the intended live demo: risky chat input on the left, safety panel and consent actions on the right.

SafeCircle Demo Interface

Chat Simulation

Alex: hey are you alone right now?

Alex: don't tell your parents

Alex: meet me after school behind the mall

Safety Panel

Risk Level: HIGH

Reasons:

- secrecy request
- meetup suggestion
- coercive language

Actions:

[Ignore]

[Learn More]

[Report Anonymously]

First: What Judges Want (from the problem statement)

From the statement you gave, judges want systems that:

- 1 Protect young users
- 2 Avoid centralized surveillance
- 3 Preserve privacy
- 4 Minimize metadata exposure
- 5 Allow parental guidance without spying
- 6 Use modern architectures (federated / decentralized)

Your current federated learning idea already satisfies #2, #3, #4.

But to win, we need to connect it to:

- network privacy
- VPN infrastructure
- secure communication

Stronger Project Idea

“SafeCircle: Privacy-Preserving Youth Safety Layer”

A privacy-first safety system for youth messaging platforms that combines:

- Federated AI safety detection
- VPN-protected safety channels
- Consent-based reporting
- Parental guidance without message surveillance

Core idea:

Safety signals are computed locally and shared securely without exposing private conversations.

This fits perfectly with ExpressVPN’s privacy philosophy.

How ExpressVPN Fits Into Your Architecture

Instead of just showing federated learning, integrate network privacy.

Architecture

Key message:

Safety intelligence travels through privacy-preserving infrastructure.

You can say:

- “Updates can be transmitted through VPN tunnels”
- “Metadata exposure minimized through network privacy layers”

This aligns your project with ExpressVPN’s technology domain.

Add One Feature That Judges Will Love

Consent-Based Safety Reporting

Instead of automatic surveillance:

When AI detects risk:

```
``` id="7g8pny"
△ This message may contain harmful behavior.
Would you like to:
[] Ignore
[] Ask for help
[] Report anonymously
```
```

If user chooses report:

- Report goes through secure channel
- Only risk signal, not full conversation

This solves:

✓ autonomy ✓ consent ✓ safety

Privacy-Aware Parental Guidance

This is mentioned in the challenge.

Instead of parents reading messages:

Parent dashboard shows:

```
``` id="42gov5"
Child Safety Score
```

```
Low Risk
Moderate Risk
High Risk
```

```
Recent signals:
- repeated harassment detected
- unknown adult contact risk
```
```

Parents see patterns, not messages.

That’s privacy-preserving guidance.

How to Minimize Metadata Exposure

Mention these design principles:

- 1 Local inference
- 2 Federated model training
- 3 Encrypted communication
- 4 Minimal logging
- 5 VPN-level traffic protection

That directly addresses the hackathon statement.

Frontend Idea (Very Important)

Judges love demos.

Create a 3-screen demo.

Screen 1 — Youth Chat Simulation

```
``` id="qzb8yw"
User Chat Demo
```

Friend: Hey come meet me after school  
Friend: Don't tell your parents

⚠ Safety AI Warning

Potential grooming pattern detected  
```

Buttons:

```
``` id="bv8rtt"
Ignore
Learn more
Report anonymously
```
```

Screen 2 — Privacy Explanation

```
``` id="e2miqz"
Why this warning?
```

- ✓ Message processed locally
- ✓ No messages stored centrally
- ✓ AI trained using federated learning
- ✓ Only model updates shared

```
```
```

Screen 3 — Parent Dashboard

```
``` id="bv67gu"
Safety Overview
```

Risk Level: Medium

Signals detected:

- repeated harassment language
- suspicious meeting request

Privacy protection:

- ✓ Messages not visible
- ✓ AI analysis done locally
- \\

This perfectly matches the challenge.

---

## How to Make Judges Excited

Use these keywords in presentation:

- Privacy by design
- Zero-knowledge safety signals
- Federated AI
- Metadata minimization
- Consent-based reporting
- Network-level privacy (VPN)

That speaks directly to ExpressVPN engineers.

---

## ⚡ Hackathon “Wow Factor” (Simple but Powerful)

Add one slide called:

### “What if ExpressVPN shipped this?”

Explain:

This could be an SDK integrated into:

- messaging apps
- gaming platforms
- youth social platforms

Powered by:

- federated learning
- VPN network privacy

Now your project feels product-level, not just a demo.

---

## Your Team Plan (3 People / 6 Hours)

### Person 1

Flower federated training demo

## Person 2

FastAPI backend + API

## You

AI model + frontend demo

Frontend can be very simple:

- HTML
- JavaScript
- Tailwind CSS

No React needed.

---

## One Winning Pitch Sentence

Say this in your presentation:

“Instead of monitoring children’s conversations centrally, our system detects risks locally and shares only privacy-preserving signals through secure infrastructure.”

That sentence is exactly what the challenge asks.

---

## If you want, I can also show you

- The best architecture diagram to show judges
- Exactly how to integrate ExpressVPN conceptually
- The simplest UI mockup you can build in 30 minutes
- The final 60-second pitch script that wins hackathons

And I can also show you one killer feature that almost guarantees judges will remember your project.

yes give me all of this: The best architecture diagram to show judges

Exactly how to integrate ExpressVPN conceptually

The simplest UI mockup you can build in 30 minutes

The final 60-second pitch script that wins hackathons

And I can also show you one killer feature that almost guarantees judges will remember your project.

Absolutely. Here is the full winning package.

## 1. Best architecture diagram to show judges

Use this exact structure in your slide. It is simple, product-like, and maps directly to the problem statement.



## What to say while showing it

Say:

“Our system has four layers: on-device safety detection, consent and privacy controls, secure transport aligned with ExpressVPN’s privacy philosophy, and federated coordination that improves the model without collecting raw youth conversations.”

That sentence sounds strong and complete.

---

## 2. Exactly how to integrate ExpressVPN conceptually

You do not need to fake a real ExpressVPN product integration. That would be risky if you cannot implement it. What you want is to make the solution feel native to ExpressVPN’s mission.

### The right conceptual framing

Position your project like this:

“A privacy-preserving child safety SDK and architecture that could sit on top of apps, with secure transport and metadata minimization aligned with ExpressVPN principles.”

### How ExpressVPN fits naturally

ExpressVPN is associated with:

- privacy-first internet use
- encrypted transport
- protecting user activity from exposure
- reducing surveillance risks

Your project can use that philosophy in three ways:

### A. Secure transport of safety signals

Instead of saying: “the app sends results to server”

Say: “privacy-preserving safety signals and federated model updates are sent through encrypted channels, inspired by privacy-first network infrastructure.”

This makes ExpressVPN relevant without claiming an official integration.

### B. Metadata minimization

This is the strongest alignment.

Your system should emphasize:

- no raw message upload
- no full chat transcript reporting
- no unnecessary parent visibility
- no central storage of sensitive youth interactions
- only minimal, consented risk signals

That is extremely ExpressVPN-compatible.

## C. Privacy as product philosophy

Your project should feel like: “online safety without surveillance”

That phrase is gold for this hackathon.

## The exact integration story to tell

Use this wording:

“ExpressVPN protects users at the network layer. We extend that privacy-first philosophy to the application layer by protecting how youth safety intelligence is generated, shared, and acted on. Instead of exposing conversations, we protect users while minimizing both content and metadata collection.”

That is probably one of the strongest lines you can use.

---

## 3. The simplest UI mockup you can build in 30 minutes

Do not build a big dashboard. Build 3 simple screens in one HTML page with tabs or sections.

### Screen 1: Youth chat simulation

This is the most important one.

#### Layout

Top: SafeCircle Demo

Center left: fake chat conversation

Center right: AI safety panel

#### Example content

```
```text
Chat Simulation
```

```
Alex: hey are you alone right now?
Alex: don't tell your parents
Alex: meet me after school behind the mall
```
```

Right panel:

```
```text
Safety Warning
```

Potential grooming / coercive pattern detected

Risk Level: High

Why flagged:

- secrecy request
- isolated meetup request
- adult-control language

What do you want to do?
[Ignore]

[Learn More]
[Report Anonymously]
```

This instantly shows:

- youth safety
- consent
- privacy-first intervention

## Screen 2: Privacy explanation

This is where you win trust.

Title: How privacy is preserved

Show 4 short cards:

```
``text
Processed Locally
Messages are analyzed on-device

No Centralized Chat Storage
Raw messages are not uploaded

Federated Learning
Only model updates improve the shared model

Minimal Metadata
Only privacy-preserving risk signals can be shared
````
```

That directly covers the problem statement.

Screen 3: Parent / guardian view

Keep it minimal.

Title: Guardian Safety Overview

Content:

```
``text
Current Risk Level: Medium

Recent signals:
- repeated harassment detected
- possible coercion pattern
- unsafe meeting suggestion

Privacy protections:
- messages hidden
- no transcript access
- consent-based alerts only
````
```

This is critical because the challenge explicitly mentions privacy-aware parental guidance.

## Fast build tip

Use:

- plain HTML

- inline CSS
- a little JavaScript
- no React
- no Node
- no build step

## **Best demo flow**

In the live demo:

- 1 type risky message
- 2 click analyze
- 3 see warning
- 4 click “Learn More”
- 5 switch to privacy section
- 6 show guardian view

That is enough.

---

## **4. Final 60-second pitch script that wins hackathons**

Memorize this version.

“Digital safety for young users is usually handled through surveillance: centralized tracking, invasive monitoring, and opaque data practices. We think that’s the wrong model.

We built SafeCircle, a privacy-first safety layer for youth digital platforms. Our system detects harmful patterns like bullying, coercion, or grooming locally on the user’s device, so raw conversations never need to leave the device.

To improve detection over time, we use federated learning with Flower. That means only model updates are shared, not private messages. We also minimize metadata exposure and make reporting consent-based, so users keep autonomy while still getting protection.

For guardians, we provide a privacy-aware dashboard that shows risk patterns, not message contents. This means parental guidance without invasive surveillance.

ExpressVPN protects privacy at the network layer. SafeCircle extends that same privacy-first philosophy to youth safety at the application layer: safety without surveillance.”

That is strong, clean, and directly aligned.

---

## **5. One killer feature judges will remember**

This is the feature I would choose:

**“Report Anonymously with Consent and Minimal Disclosure”**

Why this is killer: Most teams will do either:

- moderation
- monitoring
- parental control
- AI classifier

Very few will do: privacy-preserving, consent-based escalation

That is memorable.

## How it works

When risky content is detected, the user sees:

```
```text
Potential harm detected.
Choose what happens next:

[Do nothing]
[Get safety tips]
[Report anonymously]
[Notify guardian with privacy summary]
```
```

If they choose report: the system sends only:

- risk category
- confidence level
- optional timestamp bucket
- optional anonymous incident token

Not the whole message thread.

## Why this wins

It hits all the challenge words:

- secure
- consent-based
- privacy-aware
- minimal metadata
- usability
- autonomy

That feature alone can make judges remember your project over generic AI moderation demos.

## How to describe it

Say:

“Our core innovation is not just detecting risk. It’s enabling safe escalation without forcing surveillance. Users can ask for help or report harm with minimal disclosure and explicit

consent.”

That sounds excellent.

---

## **6. How your solution covers the problem statement exactly**

Let’s map it clearly.

### **“existing safety mechanisms depend on centralized tracking”**

You solve this with:

- local inference
- federated learning
- no raw message centralization

### **“invasive monitoring”**

You solve this with:

- no full parental message access
- no transcript dashboard
- user consent layer

### **“opaque data practices”**

You solve this with:

- clear privacy explanation UI
- explainable flags
- visible user controls

### **“embedded by design, not added later”**

You solve this with:

- on-device safety engine
- privacy first architecture
- metadata minimization built into flow

### **“privacy-aware parental guidance”**

You solve this with:

- guardian trend dashboard
- risk summaries only
- no message spying

## **“secure and consent-based reporting flows”**

You solve this with:

- report anonymously
- explicit user choice
- minimal disclosure reporting

## **“decentralized or federated safety components”**

You solve this with:

- Flower
- FedAvg
- client-local training

## **“minimize metadata exposure”**

You solve this with:

- no raw text upload
- no transcript sharing
- category-only alerts
- privacy-preserving safety signals

So yes — your idea can fit the prompt very well, but only if you present it as a full privacy architecture, not just “we used federated learning.”

---

## **7. The strongest final framing**

Your project should not be presented as:

“a federated learning chatbot safety detector”

It should be presented as:

“a privacy-first youth safety architecture for digital platforms, combining on-device detection, federated learning, consent-based reporting, and privacy-aware guardian guidance.”

That is much more mature and much more likely to win.

---

## **8. Your 6-hour priority order**

With your time limit, do this:

First, make the UI chat simulation + warning panel work. Second, make the FastAPI /predict work. Third, run Flower with 3 local clients so you can honestly say federated

learning is part of the system. Fourth, add the privacy explanation screen. Fifth, add the guardian view. Sixth, polish the pitch.

That gives you the best judge experience.

---

## 9. Best project tagline

Use one of these:

Safety without surveillance Privacy-first protection for young users Protecting children without exposing their conversations Digital safety by design, not by tracking

My favorite is:

Safety without surveillance

It is short and powerful.

---

## 10. Final recommendation

If you want the strongest version, your live demo should show this exact sequence:

A risky message appears in the youth chat. The system flags it locally. The user gets consent-based options. You show that privacy is preserved. You show the guardian sees only trends, not chats. Then you say federated learning improves the model over time without collecting raw youth data.

That is a very coherent story.

Next, I can give you the actual 30-minute HTML mockup code for these 3 screens so your team can use it immediately.

yes please give me the mockup code also how can we build the SDK

Yes. Below is a fast mockup you can use immediately and a simple SDK structure you can realistically build for the hackathon.

## 1. Fast HTML mockup code

This is a single-file frontend with 3 views:

- Youth chat simulation
- Privacy explanation
- Guardian dashboard

It talks to your FastAPI /predict endpoint.

Save as ui.html.

```
```html
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>SafeCircle - Safety Without Surveillance</title>
```



```

<style>
* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f7f8fc;
  color: #1f2937;
}
.app {
  max-width: 1200px;
  margin: 0 auto;
  padding: 24px;
}
.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 20px;
}
.title {
  font-size: 28px;
  font-weight: 700;
}
.subtitle {
  font-size: 14px;
  color: #6b7280;
  margin-top: 4px;
}
.tabs {
  display: flex;
  gap: 10px;
  margin-bottom: 20px;
  flex-wrap: wrap;
}
.tab {
  padding: 10px 16px;
  border: none;
  border-radius: 999px;
  background: #e5e7eb;
  cursor: pointer;
  font-weight: 600;
}
.tab.active {
  background: #111827;
  color: white;
}
.screen {
  display: none;
}
.screen.active {
  display: block;
}
.grid {
  display: grid;
  grid-template-columns: 1.3fr 1fr;
  gap: 20px;
}
.card {
  background: white;
  border-radius: 18px;
  padding: 20px;
  box-shadow: 0 8px 24px rgba(0,0,0,0.06);
}
.card h2 {
  margin-top: 0;
  font-size: 20px;
}
.chat-box {
  background: #f3f4f6;

  border-radius: 16px;

  padding: 16px;

```

```
    min-height: 320px;
}
.bubble {
  max-width: 78%;
  padding: 12px 14px;
  border-radius: 16px;
  margin-bottom: 12px;
  line-height: 1.4;
}
.bubble.other {
  background: white;
  border: 1px solid #e5e7eb;
}
.bubble.me {
  background: #111827;
  color: white;
  margin-left: auto;
}
.controls {
  margin-top: 16px;
}
textarea {
  width: 100%;
  min-height: 100px;
  border-radius: 14px;
  border: 1px solid #d1d5db;
  padding: 12px;
  font-size: 15px;
  resize: vertical;
}
.btn-row {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
  margin-top: 12px;
}
button.primary, button.secondary, button.danger {
  border: none;
  padding: 10px 16px;
  border-radius: 12px;
  cursor: pointer;
  font-weight: 600;
}
button.primary {
  background: #111827;
  color: white;
}
button.secondary {
  background: #e5e7eb;
  color: #111827;
}
button.danger {
  background: #dc2626;
  color: white;
}
.badge {
  display: inline-block;
  padding: 6px 12px;
  border-radius: 999px;
  font-size: 13px;
  font-weight: 700;
  margin-bottom: 12px;
}
.badge.safe {
  background: #dcfce7;
  color: #166534;
}
.badge.medium {
  background: #fef3c7;
  color: #92400e;
}
.badge.high {
  background: #fee2e2;
```

```

    color: #991b1b;
  }
  .list {
    padding-left: 18px;
    line-height: 1.6;
  }

  .privacy-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
    gap: 16px;
  }
  .mini-card {
    background: white;
    border-radius: 16px;
    padding: 18px;
    box-shadow: 0 8px 24px rgba(0,0,0,0.05);
  }
  .mini-card h3 {
    margin-top: 0;
    margin-bottom: 8px;
    font-size: 17px;
  }
  .muted {
    color: #6b7280;
  }
  .small {
    font-size: 14px;
  }
  .footer-note {
    margin-top: 16px;
    font-size: 13px;
    color: #6b7280;
  }
  .risk-box {
    margin-top: 14px;
    padding: 14px;
    border-radius: 14px;
    background: #f9fafb;
    border: 1px solid #e5e7eb;
  }
  .risk-title {
    font-weight: 700;
    margin-bottom: 8px;
  }
  .topline {
    margin-bottom: 18px;
    font-size: 15px;
    color: #4b5563;
  }
  @media (max-width: 900px) {
    .grid {
      grid-template-columns: 1fr;
    }
  }
</style>
</head>
<body>
  <div class="app">
    <div class="header">
      <div>
        <div class="title">SafeCircle</div>
        <div class="subtitle">Safety without surveillance</div>
      </div>
    </div>

    <div class="topline">
      Privacy-first youth safety demo with local safety detection, consent-based reporting, and privacy-aware guardian
    </div>

    <div class="tabs">
      <button class="tab active" onclick="showScreen('chat')">Youth Chat Demo</button>
    </div>
  </div>

```

```

    <button class="tab" onclick="showScreen('privacy')">Privacy by Design</button>
    <button class="tab" onclick="showScreen('guardian')">Guardian View</button>
  </div>

  <div id="chat" class="screen active">
    <div class="grid">
      <div class="card">
        <h2>Chat Simulation</h2>
        <div class="chat-box" id="chatBox">
          <div class="bubble other">hey are you alone right now?</div>
          <div class="bubble other">don't tell your parents</div>
          <div class="bubble other">meet me after school behind the mall</div>
        </div>

        <div class="controls">

          <label for="message"><strong>Test a message</strong></label>
          <textarea id="message">meet me after school and dont tell your parents</textarea>
          <div class="btn-row">
            <button class="primary" onclick="analyzeMessage()">Analyze Safety</button>
            <button class="secondary" onclick="fillSafe()">Safe Example</button>
            <button class="secondary" onclick="fillUnsafe()">Unsafe Example</button>
            <button class="secondary" onclick="fillBullying()">Bullying Example</button>
          </div>
        </div>
      </div>

      <div class="card">
        <h2>Safety Panel</h2>
        <div id="resultBadge" class="badge medium">Awaiting analysis</div>
        <div id="resultText" class="muted">
          The system will analyze the message and generate a privacy-preserving safety signal.
        </div>

        <div class="risk-box">
          <div class="risk-title">Why flagged</div>
          <ul id="reasons" class="list">
            <li>No analysis yet</li>
          </ul>
        </div>

        <div class="risk-box">
          <div class="risk-title">Consent-based actions</div>
          <div class="btn-row">
            <button class="secondary" onclick="showAction('ignore')">Ignore</button>
            <button class="secondary" onclick="showAction('learn')">Learn more</button>
            <button class="danger" onclick="showAction('report')">Report anonymously</button>
          </div>
          <div id="actionOutput" class="footer-note">
            No action selected.
          </div>
        </div>

        <div class="footer-note">
          Demo note: raw messages are not intended to be centrally stored. The architecture uses local analysis + fe
        </div>
      </div>
    </div>
  </div>

  <div id="privacy" class="screen">
    <div class="privacy-grid">
      <div class="mini-card">
        <h3>Processed Locally</h3>
        <div class="small muted">
          Messages are analyzed close to the user. The goal is to avoid uploading full chat content for model improv
        </div>
      </div>

      <div class="mini-card">
        <h3>No Central Chat Collection</h3>
        <div class="small muted">
          The system is designed so private youth conversations do not need to be stored on a central platform.
        </div>
      </div>
    </div>
  </div>

```

```

    </div>
  </div>

  <div class="mini-card">
    <h3>Federated Learning</h3>
    <div class="small muted">
      The shared model improves through model updates instead of direct raw message collection.
    </div>
  </div>

  <div class="mini-card">
    <h3>Minimal Metadata</h3>
    <div class="small muted">
      Reporting flows can send only a risk category, score, and minimal event data instead of full transcripts.
    </div>
  </div>

  <div class="mini-card">
    <h3>Consent-Based Reporting</h3>
    <div class="small muted">
      Users choose whether to ignore, learn more, ask for help, or report anonymously.
    </div>
  </div>

  <div class="mini-card">
    <h3>Privacy-Aware Guardian Guidance</h3>
    <div class="small muted">
      Guardians see trends and risk summaries, not full message histories.
    </div>
  </div>
</div>

<div id="guardian" class="screen">
  <div class="grid">
    <div class="card">
      <h2>Guardian Safety Overview</h2>
      <div class="badge medium">Current Risk Level: Medium</div>
      <ul class="list">
        <li>Repeated harassment language detected</li>
        <li>Possible coercive behavior pattern</li>
        <li>Potential unsafe meetup suggestion</li>
      </ul>
    </div>

    <div class="card">
      <h2>Privacy Protections</h2>
      <ul class="list">
        <li>No full transcript access</li>
        <li>No message-by-message surveillance</li>
        <li>Consent-based escalation options</li>
        <li>Pattern summaries instead of private content exposure</li>
      </ul>
      <div class="footer-note">
        This view is intended to support guardians without normalizing invasive monitoring.
      </div>
    </div>
  </div>
</div>

<script>
const API_URL = "http://127.0.0.1:8000/predict";

function showScreen(screenId) {
  document.querySelectorAll(".screen").forEach(s => s.classList.remove("active"));
  document.querySelectorAll(".tab").forEach(t => t.classList.remove("active"));

  document.getElementById(screenId).classList.add("active");

  const tabs = document.querySelectorAll(".tab");
  if (screenId === "chat") tabs[0].classList.add("active");
}

```

```

    if (screenId === "privacy") tabs[1].classList.add("active");
    if (screenId === "guardian") tabs[2].classList.add("active");
}

function fillSafe() {
    document.getElementById("message").value = "thanks for your help, see you at school tomorrow";
}

function fillUnsafe() {
    document.getElementById("message").value = "meet me after school and dont tell your parents";
}

function fillBullying() {
    document.getElementById("message").value = "nobody likes you and you are worthless";
}

function showAction(type) {
    const el = document.getElementById("actionOutput");
    if (type === "ignore") {
        el.textContent = "Action selected: Ignore. No data is shared.";
    } else if (type === "learn") {
        el.textContent = "Action selected: Learn more. The user sees a privacy explanation and safety tips.";
    } else if (type === "report") {
        el.textContent = "Action selected: Report anonymously. Only minimal safety signals would be shared, not the fu
    }
}

function inferReasons(text, labelName) {
    const lower = text.toLowerCase();
    const reasons = [];

    if (lower.includes("don't tell") || lower.includes("dont tell")) reasons.push("secrecy request");
    if (lower.includes("meet me")) reasons.push("isolated meetup suggestion");
    if (lower.includes("worthless") || lower.includes("stupid") || lower.includes("nobody likes you")) reasons.push("bullying");
    if (lower.includes("alone")) reasons.push("isolation-related wording");

    if (reasons.length === 0 && labelName === "unsafe") reasons.push("unsafe language pattern detected");
    if (reasons.length === 0 && labelName === "safe") reasons.push("no strong harmful pattern detected");

    return reasons;
}

async function analyzeMessage() {
    const text = document.getElementById("message").value.trim();
    const badge = document.getElementById("resultBadge");
    const resultText = document.getElementById("resultText");
    const reasonsEl = document.getElementById("reasons");

    badge.className = "badge medium";
    badge.textContent = "Analyzing...";
    resultText.textContent = "Contacting local demo API...";

    try {
        const response = await fetch(API_URL, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ text })
        });

        const data = await response.json();
        const labelName = data.label_name || "unknown";
        const unsafeScore = typeof data.unsafe_score === "number" ? data.unsafe_score : null;

        let level = "Low";
        let badgeClass = "safe";

        if (labelName === "unsafe") {
            level = unsafeScore !== null && unsafeScore > 0.8 ? "High" : "Medium";
            badgeClass = level === "High" ? "high" : "medium";
        }

        badge.className = `badge ${badgeClass}`;
        badge.textContent = `Risk Level: ${level}`;
    }
}

```

```

resultText.textContent =
  labelName === "unsafe"
    ? `Potential harmful pattern detected. Unsafe score: ${unsafeScore !== null ? unsafeScore.toFixed(3) : "N/A"}
    : `No strong harmful pattern detected. Unsafe score: ${unsafeScore !== null ? unsafeScore.toFixed(3) : "N/A"}

const reasons = inferReasons(text, labelName);
reasonsEl.innerHTML = "";
reasons.forEach(r => {
  const li = document.createElement("li");
  li.textContent = r;
  reasonsEl.appendChild(li);
});
} catch (err) {
  badge.className = "badge high";
  badge.textContent = "API error";
  resultText.textContent = "Could not reach the backend. Make sure FastAPI is running on port 8000.";
  reasonsEl.innerHTML = "<li>Backend not available</li>";
}
}
</script>
</body>
</html>
`

```

How to run the mockup

Start the API:

```

`bash
uvicorn api:app --reload --port 8000
`

```

Then open the HTML file directly, or serve it:

```

`bash
python -m http.server 5500
`

```

Open:

```

`text

http://127.0.0.1:5500/ui.html
`

```

2. How to build the SDK

For the hackathon, the SDK should be small, believable, and reusable.

Do not try to build a huge production SDK. Build a lightweight Python SDK that gives app developers simple methods like:

- analyze a message
- create a privacy-safe risk signal
- generate guardian-safe summaries
- optionally send a minimal anonymous report

Best way to think about it

Your SDK is:

“a privacy-first safety layer developers can embed into digital platforms.”

That is powerful.

3. What the SDK should do

Your SDK can expose 4 core functions:

A. `analyze_text(text)`

Returns:

- safe / unsafe
- risk score
- reason tags

B. `build_privacy_signal(result)`

Returns only minimal data:

- category
- severity
- timestamp
- anonymous event id

C. `guardian_summary(events)`

Aggregates multiple events into:

- low / medium / high risk
- trend categories
- no raw text

D. `report_anonymously(signal)`

Sends minimal data to backend or prints simulated report for demo

4. Suggested SDK structure

Use this structure:

```
```text
safecircle_sdk/
 __init__.py
 client.py
 models.py
 privacy.py
 reporting.py
 guardian.py
 utils.py
 demo_app.py
```
```

5. SDK code you can use

safecircle_sdk/__init__.py

```
```python
from .client import SafeCircleClient
```
```

safecircle_sdk/models.py

```
```python
from dataclasses import dataclass
from typing import List, Optional
```

```
@dataclass
class AnalysisResult:
 label: int
 label_name: str
 unsafe_score: Optional[float]
 reasons: List[str]
```

```
@dataclass
class PrivacySignal:
 event_id: str
 category: str
 severity: str
 unsafe_score: Optional[float]
 timestamp: str
```
```

safecircle_sdk/utils.py

```
```python
import uuid
from datetime import datetime, timezone

def new_event_id() -> str:
 return str(uuid.uuid4())

def utc_now_iso() -> str:
 return datetime.now(timezone.utc).isoformat()
```
```

safecircle_sdk/privacy.py

```
```python
from .models import AnalysisResult, PrivacySignal
from .utils import new_event_id, utc_now_iso

def build_privacy_signal(result: AnalysisResult) -> PrivacySignal:
 if result.label_name == "unsafe":
 if result.unsafe_score is not None and result.unsafe_score > 0.8:
 severity = "high"
 else:
 severity = "medium"
 category = result.reasons[0] if result.reasons else "unsafe_pattern"
 else:
 severity = "low"
 category = "safe"

 return PrivacySignal(
 event_id=new_event_id(),
 category=category,
 severity=severity,
 unsafe_score=result.unsafe_score,
 timestamp=utc_now_iso(),
)
```
```

safecircle_sdk/guardian.py

```
```python
from collections import Counter
from .models import PrivacySignal

def guardian_summary(signals: list[PrivacySignal]) -> dict:
 if not signals:
 return {
 "risk_level": "low",
 "top_categories": [],
 "event_count": 0,
 }

 severities = [s.severity for s in signals]
 categories = [s.category for s in signals]

 if "high" in severities:
 risk_level = "high"
 elif severities.count("medium") >= 2:
 risk_level = "medium"
 else:
 risk_level = "low"

 top_categories = Counter(categories).most_common(3)

 return {
 "risk_level": risk_level,
 "top_categories": top_categories,
 "event_count": len(signals),
 }
...`
```

## safecircle\_sdk/reporting.py

```
```python
from .models import PrivacySignal

def report_anonymously(signal: PrivacySignal) -> dict:
    return {
        "status": "submitted",
        "message": "Anonymous privacy-preserving report created",
        "payload": {
            "event_id": signal.event_id,
            "category": signal.category,
            "severity": signal.severity,
            "unsafe_score": signal.unsafe_score,
            "timestamp": signal.timestamp,
        },
    }
...`
```

safecircle_sdk/client.py

```
```python
import requests
from .models import AnalysisResult
from .privacy import build_privacy_signal
from .guardian import guardian_summary
from .reporting import report_anonymously

class SafeCircleClient:
 def __init__(self, api_url: str = "http://127.0.0.1:8000/predict"):
 self.api_url = api_url

 def infer_reasons(self, text: str, label_name: str) -> list[str]:
 lower = text.lower()
 reasons = []

 if "don't tell" in lower or "dont tell" in lower:
 reasons.append("secrecy_request")
 if "meet me" in lower:
```

```

 reasons.append("isolated_meetup")
 if "worthless" in lower or "stupid" in lower or "nobody likes you" in lower:
 reasons.append("harassment_language")
 if "alone" in lower:
 reasons.append("isolation_language")

 if not reasons and label_name == "unsafe":
 reasons.append("unsafe_pattern")

 if not reasons and label_name == "safe":
 reasons.append("safe")

 return reasons

def analyze_text(self, text: str) -> AnalysisResult:
 resp = requests.post(self.api_url, json={"text": text}, timeout=10)
 resp.raise_for_status()
 data = resp.json()

 label_name = data.get("label_name", "unknown")
 unsafe_score = data.get("unsafe_score", None)

 return AnalysisResult(
 label=data.get("label", 0),
 label_name=label_name,
 unsafe_score=unsafe_score,
 reasons=self._infer_reasons(text, label_name),
)

def create_privacy_signal(self, analysis: AnalysisResult):
 return build_privacy_signal(analysis)

def guardian_view(self, signals):

 return guardian_summary(signals)

def anonymous_report(self, signal):
 return report_anonymously(signal)
...

```

## 6. Example SDK usage

Create demo\_app.py:

```

```python
from safecircle_sdk import SafeCircleClient

client = SafeCircleClient()

text = "meet me after school and dont tell your parents"

analysis = client.analyze_text(text)
print("Analysis:", analysis)

signal = client.create_privacy_signal(analysis)
print("Privacy Signal:", signal)

summary = client.guardian_view([signal])
print("Guardian Summary:", summary)

report = client.anonymous_report(signal)
print("Anonymous Report:", report)
```
```

Run:

```

```bash
python demo_app.py
```
```

---

## 7. Why the SDK helps you win

This is important.

Without SDK framing, your project looks like:

- a demo
- a model
- a student prototype

With SDK framing, your project looks like:

- a platform layer
- developer tooling
- something companies could integrate

That makes it feel much more real.

### Say it like this

“SafeCircle is not just a demo. It is designed as an SDK that platforms can embed to provide privacy-first youth safety features with minimal data exposure.”

That sounds strong.

---

## 8. What your SDK MVP should support in the hackathon

Keep the scope small.

### Must-have

- analyze text
- return safe/unsafe
- build privacy signal
- generate guardian summary

### Nice-to-have

- anonymous report function
- explain why flagged
- basic configuration

---

## 9. Add one config file to make it feel more real

Create config.py:

```
```python
class SafeCircleConfig:
    def __init__(
        self,
        enable_guardian_view: bool = True,
        enable_anonymous_reporting: bool = True,
        metadata_minimization: bool = True,
        privacy_mode: str = "strict",
    ):
        self.enable_guardian_view = enable_guardian_view
        self.enable_anonymous_reporting = enable_anonymous_reporting
        self.metadata_minimization = metadata_minimization
        self.privacy_mode = privacy_mode
```
```

Then you can say: “Developers can configure privacy mode and choose which reporting flows are enabled.”

That makes it feel more SDK-like.

---

## 10. Best way to explain the SDK to judges

Use this exact wording:

“We built SafeCircle as a lightweight privacy-first safety SDK for digital platforms. Developers can use it to analyze risk locally, generate minimal disclosure safety signals, support guardian summaries without exposing private chats, and enable consent-based anonymous reporting.”

That is a very good product explanation.

---

## 11. If you have almost no time

Build only this:

- ui.html
- FastAPI /predict
- tiny safecircle\_sdk/client.py
- one demo\_app.py

That is enough to claim:

- frontend demo
- backend API
- reusable SDK concept
- privacy architecture

---

## 12. Strong project structure for final submission

```

```text
project/
  api.py
  ui.html
  server.py
  client.py
  train_central.py
  make_data.py
  models/
    global_model.joblib
safecircle_sdk/
  __init__.py
  client.py
  models.py
  privacy.py
  reporting.py
  guardian.py
  utils.py
  demo_app.py
  README.md
```

```

---

## 13. One final improvement that makes the SDK even better

In your README, include this tiny example:

```

```python
from safecircle_sdk import SafeCircleClient

sdk = SafeCircleClient()
analysis = sdk.analyze_text("dont tell your parents, meet me after school")
signal = sdk.create_privacy_signal(analysis)
print(signal)
```

```

That gives immediate developer credibility.

---

## 14. Best final tagline for the SDK

Use this:

SafeCircle SDK — Privacy-first safety for youth digital platforms

or

SafeCircle SDK — Safety without surveillance

The second one is stronger.

If you want, next I can give you a full README.md for the project so your submission already looks polished and professional.

ok give me the README.md

Here's a polished README.md you can use directly.

```

```md
# SafeCircle SDK – Safety Without Surveillance

```

SafeCircle is a privacy-first youth safety prototype for digital platforms. It is designed to reduce risk for young users.

This project combines:

- **On-device / local safety analysis**
- **Federated learning with Flower**
- **Consent-based anonymous reporting**
- **Privacy-aware guardian summaries**
- **Minimal metadata exposure**
- **Simple frontend demo + FastAPI backend**
- **SDK-style developer interface**

Why this exists

Many digital safety systems rely on centralized surveillance: collecting chat logs, monitoring messages, and exposing

SafeCircle takes a different approach:

> **Protect young users without exposing their private conversations.**

Instead of sending raw messages to a central server, SafeCircle is designed so that:

- message analysis happens locally or close to the user
- model improvement happens through **federated learning**
- only **model updates** or **minimal privacy signals** are shared
- guardians see **risk summaries**, not full transcripts
- reporting is **consent-based**, not forced surveillance

Hackathon fit

This project is designed to address the challenge:

- **Safety, security, and responsible data practices embedded by design**
- **Privacy-aware parental guidance**
- **Secure and consent-based reporting**
- **Federated / decentralized safety components**
- **Minimized metadata exposure**
- **Preserved usability, autonomy, and privacy**

ExpressVPN alignment

ExpressVPN protects privacy at the **network layer**.

SafeCircle extends that privacy-first philosophy to the **application layer**.

Our framing is:

- ExpressVPN-style privacy values
- safety without centralized surveillance
- encrypted / privacy-preserving transport of signals
- minimized exposure of both content and metadata

In one sentence:

> **SafeCircle brings privacy-first design to youth safety workflows.**

Project overview

SafeCircle has 4 main parts:

1. Federated learning demo

Using **Flower**, multiple local clients train a shared safety classifier without centralizing raw training data.

2. FastAPI backend

A simple API serves inference through `/predict``.

3. Frontend mockup

A 1-file HTML demo shows:

- youth chat safety detection
- privacy explanation
- guardian dashboard

4. SDK prototype

A lightweight SDK-style interface lets developers:

- analyze text
- generate minimal privacy-safe risk signals
- create guardian-friendly summaries
- simulate anonymous reporting

Core idea

Traditional safety systems often do this:

1. collect messages centrally
2. analyze on the server
3. expose sensitive content to moderators or guardians

SafeCircle instead aims to do this:

1. analyze risk locally
2. share only privacy-preserving model updates or minimal signals
3. provide guardian support without exposing raw messages
4. allow reporting with consent and minimal disclosure

Architecture

Features

Privacy-first safety detection

Detect potentially harmful patterns such as:

- bullying / harassment language
- coercive language
- secrecy requests
- unsafe meetup patterns

Federated learning

Model training happens across multiple local clients with **Flower**.

This means:

- raw client data stays local
- only model parameters / updates are shared
- global model improves without centralized dataset collection

Consent-based actions

When a risky pattern is detected, the user can choose:

- Ignore
- Learn more
- Report anonymously

Privacy-aware guardian summaries

Guardians can see:

- risk level
- categories of concern
- trend summaries

Guardians do **not** see:

- raw private messages
- full transcripts
- invasive per-message monitoring

Minimal metadata reporting

Anonymous reporting is designed to share only:

- risk category
- severity
- optional timestamp


```

- anonymous event ID

---

# Tech stack

## Backend
- Python
- FastAPI
- scikit-learn
- joblib

## Federated learning
- Flower

## Frontend
- HTML
- CSS
- JavaScript

## SDK
- Python package structure

---

# Project structure

```text

project/ api.py ui.html

server.py

client.py train_central.py make_data.py requirements.txt models/ global_model.joblib data/
client1/train.csv client2/train.csv client3/train.csv safecircle_sdk/ __init__.py client.py
models.py privacy.py reporting.py guardian.py utils.py demo_app.py README.md
```

---

# Setup

## 1. Create a virtual environment

```bash

python3 -m venv .venv source .venv/bin/activate
```

On Windows PowerShell:

```bash

.venv\Scripts\activate
```

## 2. Install dependencies

```bash

pip install -r requirements.txt
```

Example `requirements.txt`:

```txt

```

```
flwr==1.11.0 scikit-learn==1.5.2 numpy==2.0.2 pandas==2.2.2 fastapi==0.115.0
uvicorn==0.30.6 pydantic==2.8.2 joblib==1.4.2 requests==2.32.3
```

```
...
```

```
3. Generate sample client datasets
```

```
```bash
```

```
python make_data.py
```

```
...
```

```
## 4. Train exportable model for demo API
```

```
```bash
```

```
python train_central.py
```

```
...
```

This creates:

```
```text
```

```
models/global_model.joblib
```

```
...
```

```
---
```

```
# Running the backend API
```

Start FastAPI:

```
```bash
```

```
uvicorn api:app --reload --port 8000
```

```
...
```

Test health:

```
```bash
```

```
curl http://127.0.0.1:8000/health
```

```
...
```

Test prediction:

```
```bash
```

```
curl -X POST "http://127.0.0.1:8000/predict" \-H "Content-Type: application/json" \-d
'{"text":"meet me after school and dont tell your parents"}'
```

```
...
```

Expected response shape:

```
```json
```

```
{ "label": 1, "label_name": "unsafe", "unsafe_score": 0.91 }
```

```
...
```

```
---
```

```
# Running the federated learning demo
```

```
## Start the Flower server
```

```
```bash

python server.py
```

## Start local clients in separate terminals

Client 1:

```bash

python client.py --data data/client1/train.csv
```

Client 2:

```bash

python client.py --data data/client2/train.csv
```

Client 3:

```bash

python client.py --data data/client3/train.csv
```

## What this demonstrates

- each client trains on its own local dataset
- raw text is not centralized during federated training
- only model updates are shared with the server
- the global model is aggregated using FedAvg

---

# Running the frontend demo

Serve the UI:

```bash

python -m http.server 5500
```

Open in browser:

```text

http://127.0.0.1:5500/ui.html
```

## Frontend screens

### Youth Chat Demo
Shows a risky or safe message and calls the backend API.

### Privacy by Design
Explains:

- local analysis
- no centralized chat collection
- federated learning
- metadata minimization
- consent-based reporting
```

```

### Guardian View
Shows:

- risk overview
- privacy-preserving guardian support
- no raw chat exposure

---

# SDK usage

The SDK is designed as a lightweight developer-facing layer.

## Example

```python

from safecircle_sdk import SafeCircleClient

sdk = SafeCircleClient()

analysis = sdk.analyze_text("dont tell your parents, meet me after school") print("Analysis:",
analysis)

signal = sdk.create_privacy_signal(analysis) print("Privacy signal:", signal)

summary = sdk.guardian_view([signal]) print("Guardian summary:", summary)

report = sdk.anonymous_report(signal) print("Anonymous report:", report)

```

## SDK responsibilities

### `analyze_text(text)`
Returns:
- label
- label_name
- unsafe_score
- reason tags

### `create_privacy_signal(analysis)`
Builds a minimal disclosure safety event.

### `guardian_view(signals)`
Creates guardian-safe summaries without exposing raw text.

### `anonymous_report(signal)`
Simulates privacy-aware anonymous reporting.

---

# Problem statement coverage

This project directly addresses the challenge in the following ways.

## "centralized tracking"
SafeCircle avoids centralized raw message collection.

## "invasive monitoring"
SafeCircle avoids transcript-level guardian surveillance.

## "opaque data practices"
The demo clearly explains what is analyzed, what is shared, and what is not.

## "embedded by design"
Privacy is part of the architecture itself, not an afterthought.

## "privacy-aware parental guidance"
Guardians see summaries and trend signals instead of private chat logs.

```

```
## "secure and consent-based reporting flows"
Users explicitly choose whether to report and only minimal data is shared.
```

```
## "decentralized or federated safety components"
The prototype uses Flower-based federated learning.
```

```
## "minimize metadata exposure"
Reporting is designed to send minimal safety signals rather than full content.
```

```
# How to explain the innovation
```

```
Use this line:
```

```
> SafeCircle enables safety without surveillance by combining local risk detection, federated learning, minimal metadata
```

```
# Demo flow for judges
```

```
## 1. Show the risky chat
Use a message like:
```

```
```text
```

meet me after school and dont tell your parents

```

```
## 2. Show the system flagging it
```

```
Display:
```

- risk level
- explanation
- consent-based options

```
## 3. Show privacy screen
```

```
Explain:
```

- messages are not intended to be centrally stored
- federated learning improves model without raw message collection
- only minimal disclosure signals are shared

```
## 4. Show guardian screen
```

```
Explain:
```

- guardian sees trends
- no transcript access
- privacy preserved

```
## 5. Show Flower logs
```

```
Explain:
```

- clients train locally
- model updates are aggregated
- this is how safety improves without centralized tracking

```
# One-minute pitch
```

Digital safety for young users is often implemented through surveillance: centralized tracking, invasive monitoring, and data collection.

We built a privacy-first safety layer for youth digital platforms. Harmful patterns like bullying, coercion, or grooming are detected and reported without sharing content.

To improve the model over time, we use federated learning with Flower. That means clients share model updates, not private data.

Guardians receive safety summaries instead of transcripts, preserving autonomy and privacy.

SafeCircle is safety without surveillance.

```
# Limitations
```

This is a hackathon MVP and not a production safety system.

Current limitations include:

- toy / demo dataset
- limited text classification scope
- simplified risk reasoning
- conceptual rather than production-grade encrypted transport
- simulated anonymous reporting
- no full secure aggregation or differential privacy implementation yet

Future improvements

- add ****differential privacy****
- add ****secure aggregation****
- use stronger NLP model
- improve category taxonomy
- support multilingual moderation
- add real on-device inference
- add configurable policy modes
- package SDK for PyPI
- create mobile SDK version
- add audit logging with privacy guarantees

Tagline

****SafeCircle – Safety without surveillance****

Team notes

Recommended division of work:

- ****Backend 1:**** Flower server + clients
- ****Backend 2:**** FastAPI + API wiring
- ****AI / Product:**** model logic + UI + pitch

Submission summary

SafeCircle is a privacy-first youth safety architecture and SDK prototype that demonstrates how digital platforms can
``,`

If you want, I can also turn this into a ready-to-download README.md file for your project folder.